

Generating Structured Text and Rust from Natural Language via a Compact Abstract Syntax Tree and Lint-Guided Self-Repair

Wordson Robert^{1,*}, C.Jerin Mahibha^{2,*}

¹Indian Institute of Science Education and Research, Kolkata

²Meenakshi Sundararajan Engineering College, Chennai

Abstract

We describe our system for the FIRE 2026 NL2PLC shared task, which maps natural language to IEC 61131-3 Structured Text (Task A) and to Rust (Task B). A LoRA-tuned Qwen2.5-Coder model (7B or 1.5B) emits a compact JSON abstract syntax tree that a linter and a scan-cycle simulator check; problems are fed back (up to three attempts), and deterministic printers render the tree as Structured Text and Modbus-based Rust. Analysing the 30 submitted test outputs with an independent front-end, interface and timing fidelity measures and 169 behavioural checks derived from the task texts, we find that the 7B model reproduces task interfaces almost perfectly but behaviour poorly: the primary system earns 20 of the 91 checks that require a program to act, and only 2 tasks pass all of their checks. Repair raised verifier-clean programs from 15 to 18 but behavioural credit only from 18 to 20, whereas the 7B model beat the 1.5B model on 9 tasks and lost on none ($p = 0.004$), and the Modbus stub, not repair, enables all 18 of 30 Rust compilations.

Keywords

PLC code generation, IEC 61131-3, Structured Text, Rust, LoRA, self-repair

1. Introduction

Programmable logic controllers (PLCs) run much of the world’s industrial control logic, and their programs are written in the IEC 61131-3 languages [1, 2], of which Structured Text (ST) is the main textual one. Writing that logic means handling seal-ins, interlocks, timers, counters and edge detection under scan-cycle semantics, where the whole program re-runs every few milliseconds. That makes PLC code a natural but unforgiving target for code LLMs [3, 4, 5, 6, 7, 8, 9]. The FIRE 2026 AI4Industrial Automation task [10] asks systems to turn short specifications, each listing named I/O points with IEC addresses, into ST (Task A) and Rust (Task B).

Two properties shaped our design. The setting is extremely low-resource: we had a few hundred examples, and code LLMs perform much worse on rare languages [11, 12]. And many PLC bugs are cheap to detect mechanically: an input never read, an input written to, an output that can never change. We therefore separate *what the program does* from *how it is spelled*. The model writes a compact JSON abstract syntax tree (AST); deterministic code checks that tree, feeds problems back to the model, and prints the result as ST or Rust. Greedy decoding makes our single-pass configuration identical to the first repair attempt, so the effect of repair is directly measurable.

We also analyse the submitted files directly, with an independent ST front-end, interface and timing measures, and behavioural probes. Code, logs and analysis: https://github.com/WordsonRobert/AI4Industrial_Automation-FIRE2026; LoRA adapters on request.

2. Related Work

LLMs for PLC code. Chat models can produce plausible IEC 61131-3 logic for textbook-style tasks, though it needs expert review [13]; retrieving function-block libraries helps ground it [14].

Forum for Information Retrieval Evaluation, December 17–20, 2026, India

*Corresponding author.

✉ wr24ms190@iiserkol.ac.in (W. Robert); jerinmahibha@msec.ac.in (C.Jerin Mahibha)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

LLM4PLC [15] closes the loop with a syntax checker, a compiler and a model checker, and Agents4PLC [16] distributes these roles across agents, building on a long tradition of model checking PLC software [17, 18]. We differ in generating an AST rather than text, and in verifying it against the task’s own I/O list before printing.

Code LLMs, adaptation and synthetic data. Open code models [5, 6, 7, 8, 9] are uneven across languages [11], and synthesised data helps low-resource ones [12]. We use LoRA [19], one of several parameter-efficient methods [20, 21], and describe scraped programs with DeepSeek-V3 [22], as in Self-Instruct, Evol-Instruct and OSS-Instruct [23, 24, 25].

Structured and constrained generation. Decoding syntax trees instead of tokens dates to syntactic neural models and abstract syntax networks [26, 27]; with LLMs the same guarantee comes from constrained decoding, via incremental parsing [28], constrained semantic decoding [29], formal grammars [30] or token masks [31]. We do not constrain decoding, and all four of our 7B parse failures are of the kind it prevents (Section 4).

Self-repair and verification. Self-Debugging, Self-Refine and Reflexion feed execution results or critiques back to the model [32, 33, 34]. Careful studies find the gains modest and limited by feedback quality [35], and nearly absent without an external signal [36]. An alternative is to sample many candidates and select with tests or learned verifiers [37, 38, 39].

Evaluating code. The official metric mixes cosine, structural and program-dependence-graph similarity [40], akin to CodeBLEU and data-flow-aware models [41, 42]; such metrics reward similarity to a reference, and even test-based evaluation overstates correctness [43].

3. Methodology

For this task we are required to generate Structured Text (ST) and Rust code from natural language descriptions. Our approach is to first generate a compact abstract syntax tree (AST) representation of the given natural-language description of the task using an LLM, and then convert the AST into the required target language using a deterministic printer. We note that the LLM never produces code, but rather an AST representation of the code, which is then deterministically printed. We have a linter and a simulator to check the generated AST for correctness and provide feedback to the LLM for self-repair. Once the AST is verified to be correct, we use a deterministic printer to convert the AST into the required target language (ST or Rust). The overall architecture of our approach along with the preprocessing steps are shown in Figure 1.

3.1. Dataset and Preprocessing

For the AST generation task, we train Qwen2.5-Coder-7B-Instruct and Qwen2.5-Coder-1.5B-Instruct [9] with the training data provided by the organizers, together with other permissively licensed open-source PLC projects that we scraped to extend the training corpus.

More concretely, we scraped relevant PLC programs containing IEC 61131-3 Structured Text code licensed under MIT, Apache-2.0, BSD-2-Clause, BSD-3-Clause, ISC, Unlicense, and CC0-1.0 from public GitHub repositories. All accepted PLC program files were first separated into individual program units, after which template and non-program noise was removed. Each program was represented as a structured control/data-flow AST capturing declarations, statements, expressions, function-block calls, and control-flow constructs. Natural-language descriptions were generated for the resulting program units via DeepSeek-V3 [22]. The final AST representation contains 18 node types spanning program and variable declarations, assignments, function-block calls, conditional and loop constructs, returns, exits, identifiers, literals, operators, function calls, array indexing, and member access.

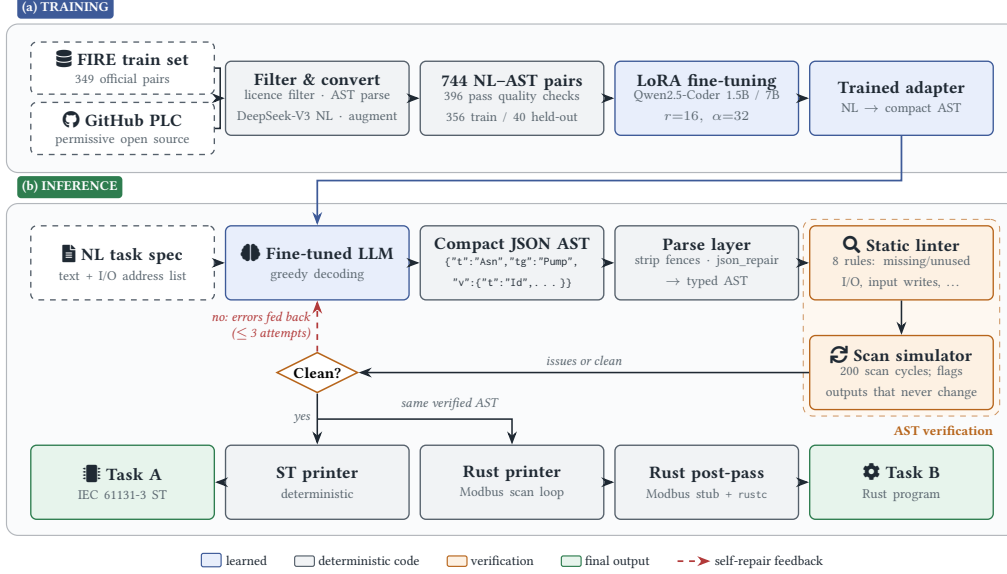


Figure 1: Overall architecture. (a) LoRA fine-tuning on (NL, compact AST) pairs from the FIRE training set and filtered open-source PLC code. (b) The model emits a compact JSON AST that is parsed, linted and simulated, with up to three attempts; the final AST is printed as ST (Task A) and Rust (Task B).

Data augmentation and merging. To increase variation without requiring additional source programs, accepted programs were augmented through systematic renaming and variable/literal substitution. These transformations preserve the underlying control-logic structure while varying I/O names, tag names, and literal values.

The augmented open-source examples were then combined with the 349 examples in the official FIRE 2026 training set. All examples were converted to the same internal representation before merging, and near-duplicate examples were removed. The resulting corpus contains 744 NL-to-AST training pairs of the form (natural-language specification, compact AST), of which 349 originate from the official FIRE training set and 395 from the additional corpus and augmentation pipeline.

3.2. LoRA fine-tuning

We freeze the parameters of the native Qwen2.5-Coder-7B-Instruct and Qwen2.5-Coder-1.5B-Instruct models and attach low-rank adaptation (LoRA) [19] modules to the q_{proj} , k_{proj} , v_{proj} , o_{proj} , $gate_{proj}$, up_{proj} , and $down_{proj}$ projection layers. We use a LoRA rank of 16, scaling factor $\alpha = 32$, and dropout of 0.05, enabling the models to learn the mapping from natural-language PLC specifications to the compact AST representation while keeping the pretrained model parameters fixed.

The 744 total corpus pairs were subject to further quality checks, and the 396 pairs that survived the filtering process were split into 356 training and 40 held-out evaluation examples using a fixed 90/10 split with random seed 42. Training was performed for three epochs using an effective batch size of 16, obtained from a per-device batch size of 1 with 16 gradient-accumulation steps, and a learning rate of 2×10^{-4} . We used bfloat16 precision, gradient checkpointing, and a maximum sequence length of 1536 tokens. Training loss and held-out evaluation metrics were monitored throughout training to assess convergence and detect potential overfitting.

3.3. AST generation and verification

The fine-tuned model receives a natural-language description of the PLC control task and is prompted to generate the corresponding compact AST in JSON format. The prompt explicitly specifies the AST schema, including the permitted node types and their compact field representations, while constraining

the model to output only the JSON AST without explanatory text, Markdown formatting, or Structured Text.

The model uses greedy decoding with a maximum of 2560 newly generated tokens. The generated output is first stripped of Markdown code fences, if present, and parsed as JSON. If standard JSON parsing fails, a JSON-repair procedure is applied before the resulting object is converted into the internal typed AST representation. Outputs that cannot be parsed are marked as generation failures.

The resulting AST is then subjected to two verification stages. First, a rule-based static analyser checks structural and data-flow constraints, including missing required I/O, unused I/O, assignments to input variables, dead overrides, impossible conditions, undeclared variables, and invalid function-block usage. Second, the AST is evaluated using a lightweight PLC scan-cycle simulator to identify dynamic issues such as outputs that remain unchanged during execution. The simulator maintains state across scans and models PLC constructs including timers, counters, and edge detectors.

If either verification stage identifies an issue, the model's previous AST output is appended to the conversation together with a diagnostic feedback message describing the detected issues, and the model is prompted to generate a corrected AST. The process is repeated for up to three attempts, with each iteration consisting of generation, AST parsing, static analysis, and simulation. A clean result terminates the process; otherwise, the last successfully parsed AST is retained.

3.4. AST to Structured Text conversion

Following AST generation and verification, the final AST is deterministically converted into IEC 61131-3 Structured Text (ST) using a dedicated AST-to-ST printer. The printer traverses the typed AST and generates the corresponding ST program without further language-model generation. Program declarations are emitted using a PROGRAM header followed by VAR_INPUT, VAR_OUTPUT, and VAR declaration blocks as required by the variable scopes represented in the AST. Statements and expressions are then recursively converted into their corresponding Structured Text syntax. Parentheses are inserted where necessary to preserve the precedence of the original expression tree.

The printer also handles formatting and indentation deterministically, with indentation applied on a per-line basis to maintain valid block structure. Since the AST retains the original IEC I/O addresses, these addresses remain available in the intermediate representation during subsequent processing.

Limitations. In the current implementation, the Structured Text printer does not emit the AT address binding in the generated declaration. For example, a variable represented internally with an address such as %QX0.0 is printed as `Fill_Pump : BOOL;` rather than `Fill_Pump AT %QX0.0 : BOOL;`. Thus, the generated ST preserves the variable names and types but does not currently materialize the IEC address bindings in the output.

3.5. AST to Rust conversion

For Task B, the same verified AST is deterministically translated into Rust using a dedicated AST-to-Rust printer. Unlike the AST generation stage, this conversion does not involve further language-model generation. The printer maps the language-independent AST into a Rust program following the execution structure of the FIRE reference implementations, consisting of a Modbus client, a cyclic control loop that reads inputs, evaluates the generated control logic, writes outputs, and pauses for 20 ms between scans. IEC bit-addressed inputs and outputs are mapped to Modbus coils, while word/register addresses are mapped to Modbus holding registers.

The printer translates common PLC function blocks into equivalent Rust state-update logic. In particular, TON timers are represented using scan counters based on the 20 ms scan interval, while CTU and R_TRIG are also supported. Function blocks not supported by the Rust backend are emitted as comments rather than being fully translated.

Modbus post-processing. The generated Rust programs initially reference the external modbus crate. Since the evaluation uses standalone `rustc` compilation rather than a Cargo-managed dependency environment, these imports would otherwise fail during compilation. The Rust post-processing pass therefore performs a static patching and compilation step. When a generated program contains a Modbus import, the pass prepends a self-contained modbus module implementing an in-memory representation of 256 coils and 256 holding registers, together with the subset of the client interface required by the generated programs. It then checks for missing helper functions, identifies certain dead overrides and hard-coded scan-count timer thresholds, and invokes `rustc -edition 2021` as a compilation gate.

In the reported runs, the Modbus stub was the only post-processing transformation that actually modified the generated files; the remaining checks were used only for detection and reporting.

Limitations. The Rust backend provides only partial coverage of the PLC language and function-block semantics. The current implementation explicitly supports TON, CTU, and R_TRIG, while other function blocks such as TP and TOF are not fully translated. The printer also uses scan-count approximations for timers and does not implement a complete IEC type system, which can lead to Rust type mismatches during compilation. In addition, certain source constructs and addressing formats remain unsupported, including the AI : /DI : addressing notation present in one test instance.

4. Results and Discussion

Configurations and evaluation. We compare three submitted configurations on the 30 test tasks: *7B single pass*, *7B + repair* (our primary system) and *1.5B + repair*. With greedy decoding, the single-pass outputs match the first attempt of *7B + repair*, so the first pair isolates repair and the second model size. In the official evaluation, which measures similarity to the organisers’ reference programs, our team ranked 8th of 8 on Task A (63.21) and 6th of 7 on Task B (53.20); as this is one score per team, we analyse the submitted files themselves. Besides the pipeline’s verdicts (parseable AST; *clean*: no lint or simulator finding), we measure (i) *front-end validity*: parsing and name resolution by an independent interpreter for the ST subset our printer emits (no type checking), and `rustc` for Rust; (ii) *interface fidelity*: whether each I/O point of the task’s address list is declared under its name, in the correct direction, with an address-consistent type, and is referenced; (iii) whether each stated duration appears as a time literal; and (iv) *behaviour*: per task, an input scenario with checks restricted to behaviour the text states (169 checks; momentary normally open buttons; 20 ms scan; outputs that are named but not described are not checked), all passed by hand-written reference programs. A program that never drives an output passes 78 checks (every “stays off” condition), so we score the other 91 *active* checks and credit one only if the same output also passes every check that requires it to be off; a program that drives every output permanently on passes 50 active checks but earns 2. Active checks fall into *memory* (26: seal-ins, latched faults, hysteresis), *timing* (34: delays, pulses, sequences, counting) and *direct* responses (31). Paired tests are exact McNemar or sign tests over tasks (Table 1).

The interface is reproduced reliably, the behaviour is not. In the parsed 7B programs every required I/O point is declared under its name and in the correct direction, except one that the ST printer dropped (task 4, below), though six are never referenced. 118 of the 121 Rust I/O points of *7B + repair* use the Modbus number that the task’s address prescribes under our mapping; the only slip drops a digit from all three addresses of task 26 (`%IX100.0` becomes `%IX10.0`). Behaviour is much weaker. On all 169 checks the three configurations score 77, 78 and 67, against 78 for the program that does nothing; the primary system earns 20 of the 91 active checks, and only tasks 5 and 9 pass all of their checks (Figure 2). Direct responses succeed most often (14/31), remembered state (3/26) and timing (3/34) rarely; no configuration keeps a hysteresis output in its dead band. Typical failures are start buttons that act only while held (e.g. tasks 3, 26), on-delay timers where a pulse or sequence is asked for, and unit errors such as T#10s for the “fixed 10 minutes” of task 17.

Table 1

Measurements on the 30 test tasks. Interface counts cover the required I/O of parsed programs. [†]Stub applied post hoc; without it, no printed Rust program compiles.

	7B single pass	7B + repair	1.5B + repair
Generation calls / parseable AST	30 / 26	58 / 27	69 / 30
Clean (lint and simulation)	15	18	12
ST passes the front-end check	24	25	26
Required I/O declared	120/121	125/126	140/151
correct direction / consistent type / referenced	120 / 116 / 113	125 / 121 / 119	137 / 133 / 129
Stated durations present as time literals	10/12	10/12	12/14
Active checks earned of 91 (tasks passing all checks)	18 (2)	20 (2)	4 (0)
memory (26) / timing (34) / direct (31)	3 / 3 / 12	3 / 3 / 14	0 / 1 / 3
Rust I/O at the Modbus number our mapping prescribes	113/116	118/121	137/146
Rust compiles (rustc, with Modbus stub)	18 [†]	18	18

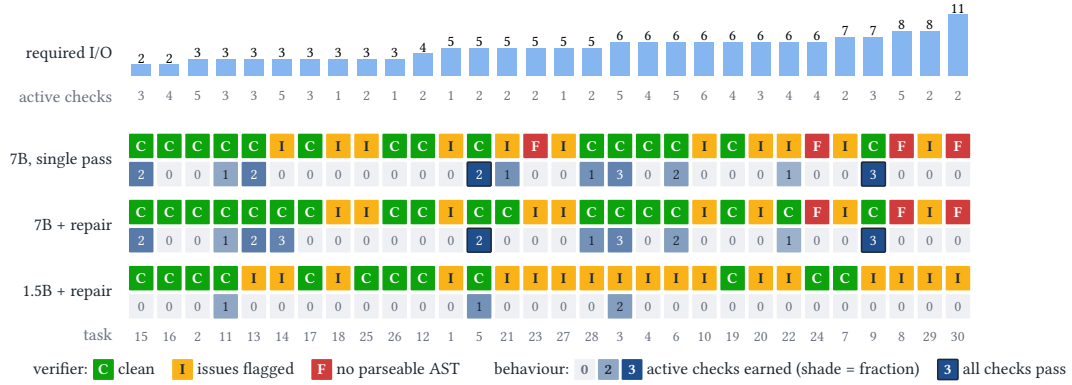


Figure 2: Per-task results, sorted by required I/O. Per configuration, the upper row gives the verifier’s verdict and the lower row the active checks earned (shade: fraction); outlined cells pass all checks.

A clean verdict is a weak indicator of correctness. Clean 7B + repair programs earn 33% of their active checks and flagged ones none: the verdict carries signal, but most clean programs violate the specification. The traffic light of task 2, clean in all configurations, uses pulse timers that all fire at start-up, so all lamps switch on together and stay dark after five seconds. Nor does verification of the tree fully carry over to the printed code: two of the 18 clean ST files fail the front-end check. In task 4 the model gave the counter output a scope label (memory) that the ST printer silently drops, although the Rust printer emits it (the same drop breaks four 1.5B programs and removes five of their required I/O points), and task 21 contains `IdStop_Button`, because the model put the node code `Id` in an operator slot that no lint rule inspects.

Repair fixes what the verifier sees, not the behaviour. Repair changed 8 of the 30 7B programs and raised clean programs from 15 to 18 (3 gained, 0 lost; $p = 0.25$), but earned checks only from 18 to 20. Its fixes are local (a dead override in task 22, an input write in task 14, a parse failure in task 23), and only task 14 gained behavioural credit. Task 21 lost its credit, showing the loop optimising the verifier rather than the task: to silence an unused-`Stop_Button` warning, the model first deleted the button and then added it to the start condition through the invalid operator above. Returns diminish quickly (attempts two and three add 2 and 1 clean 7B programs, 3 and 0 for 1.5B), never-changing 7B outputs stay at nine, and the 1.5B model deletes required I/O to suppress unused-I/O findings (3→6), in line with reports that self-repair is bounded by its feedback [35, 36].

Model size matters more than repair. Against 1.5B + repair, 7B + repair earns more active checks on 9 tasks and fewer on none ($p = 0.004$); clean counts point the same way (18 vs. 12; $p = 0.15$). The 1.5B model always yields a parseable AST, but many of its outputs are on where they should be off: it passes 14 active checks yet earns only 4. It also omits 6 required I/O points, declares 3 in the wrong direction (e.g. a position sensor as an output), and misses or mis-binds 9 of 146 Rust I/O points. Of the four 7B parse failures, three are valid JSON drifting towards familiar vocabulary (FALSE as a node type, an invented Mod node, a lowercase id) and one is malformed JSON; constrained decoding prevents all four [28, 29, 30, 31]. As the configurations fail on different tasks, picking a clean one per task would give 21 clean programs, a simple verifier-based selection [37, 38, 39].

The Modbus stub, not repair, enables Rust compilation. Every printed Rust program imports the external modbus crate, so none compiles standalone; with the 48-line stub 18 do, with or without repair (which gained task 14 and lost task 21). Of the 12 remaining 7B + repair failures, 3 lack an AST, 6 stem from printer gaps (untranslated TP timers and counter outputs, MOD, INT/REAL mixing) and 3 from the model (the invalid operator of task 21; word outputs declared BOOL in tasks 27 and 29). The address analysis also exposes a design error that rustc cannot see: in 24 task statements an input and an output share a number (e.g. %IX0.0 and %QX0.0), and since our printer reads inputs from the coil and holding-register tables instead of the discrete-input and input-register tables [44], each such pair collides on one Modbus address. Finally, the compact AST is $4.2\times$ longer than the ST it encodes and only 10% shorter in tokens than a verbose serialisation; its value is checkability, not brevity.

5. Conclusions and Future Work

In our system, a LoRA-tuned Qwen2.5-Coder writes only a compact JSON tree that is linted, simulated, repaired and printed as ST and Rust. Measured on the submitted outputs, the 7B model reproduces the task interface almost perfectly (names, directions, types and address numbers of the required I/O) but not behaviour over time: seal-ins, latched faults, hysteresis, delays and sequences account for most failed checks, and only two of thirty tasks pass every behavioural check.

Three lessons follow for LLM-based PLC code generation. First, a verifier that checks structure is a weak proxy for correctness. Clean programs earned only a third of their active checks, and the repair loop, which optimises that verifier, improved the verdict without improving behaviour, and once satisfied a rule in a way that broke the program; repair feedback should therefore come from the specification, not only from generic rules. Second, model capacity mattered more than iteration: the 7B model behaved better than the 1.5B model on 9 tasks and worse on none, although the smaller model always produced a parseable tree. Third, the deterministic components shape the easily measured outcomes, and their defects are the ones the pipeline missed: without the Modbus stub no Rust program compiles, while a dropped variable scope, the missing AT bindings and a shared Modbus address space escaped the linter, the simulator and rustc. Verifying an intermediate representation is only as strong as the guarantee that the printers preserve it.

Our analysis has limitations: the probes are hand-written, cover only stated behaviour and resolve ambiguities by convention (normally open buttons, photo-eyes true when blocked, word outputs carrying the set-point), so they indicate rather than prove correctness; with 30 tasks only large effects are detectable (p -values are uncorrected); and the official scores cannot be attributed to configurations.

Future work follows from these findings, in order of expected payoff: (1) derive executable scenarios from the specification, as we did post hoc, and use them as repair feedback and to select among sampled candidates [38, 39]; (2) make the printers total and checked (emit AT bindings, reject unknown scopes, validate operators, read inputs from the discrete-input and input-register tables, translate TP, TOF, CTD and numeric types in Rust); (3) constrain decoding to the AST schema [30, 31], which would have prevented all four 7B parse failures; (4) enrich the training data with seal-ins, timers, hysteresis and step sequences, the patterns the models fail on most; and (5) model-check safety properties such as mutually exclusive motor outputs [17, 18].

Acknowledgments

We thank the organizers of the FIRE 2026 AI4Industrial Automation task for the task, the data and their prompt answers, and the authors of the open-source PLC projects in our training corpus.

Declaration on Generative AI

During the preparation of this work, the author(s) used Claude (Anthropic) to help draft and edit the text of this working note and to help write and debug parts of the pipeline code. After using this tool, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the publication's content. DeepSeek-V3 was used only to generate training descriptions (Section 3.1).

References

- [1] International Electrotechnical Commission, IEC 61131-3:2013 Programmable Controllers – Part 3: Programming Languages, International Standard, 2013. URL: <https://webstore.iec.ch/publication/4552>.
- [2] K.-H. John, M. Tiegelkamp, IEC 61131-3: Programming Industrial Automation Systems, 2nd ed., Springer, 2010. doi:10.1007/978-3-642-12015-2.
- [3] M. Chen, et al., Evaluating large language models trained on code, 2021. arXiv:2107.03374.
- [4] J. Austin, et al., Program synthesis with large language models, 2021. arXiv:2108.07732.
- [5] E. Nijkamp, et al., CodeGen: An open large language model for code with multi-turn program synthesis, in: Proc. ICLR, 2023. arXiv:2203.13474.
- [6] R. Li, et al., StarCoder: May the source be with you!, Trans. Mach. Learn. Res. (2023). arXiv:2305.06161.
- [7] B. Rozière, et al., Code Llama: Open foundation models for code, 2023. arXiv:2308.12950.
- [8] D. Guo, et al., DeepSeek-Coder: When the large language model meets programming – the rise of code intelligence, 2024. arXiv:2401.14196.
- [9] B. Hui, et al., Qwen2.5-Coder technical report, 2024. arXiv:2409.12186.
- [10] B. Chatterjee, et al., FIRE 2026 shared task on AI4Industrial Automation: NL2PLC code generation and understanding, Task website, 2026. URL: <https://fire2026-ai4industry.github.io/fire2026-nl2plc/>.
- [11] F. Cassano, et al., MultiPL-E: A scalable and polyglot approach to benchmarking neural code generation, IEEE Trans. Softw. Eng. 49 (2023). doi:10.1109/TSE.2023.3267446.
- [12] F. Cassano, et al., Knowledge transfer from high-resource to low-resource programming languages for code LLMs, Proc. ACM Program. Lang. (OOPSLA) 8 (2024) 677–708. doi:10.1145/3689735.
- [13] H. Koziolok, S. Grüner, V. Ashiwal, ChatGPT for PLC/DCS control logic generation, in: Proc. IEEE ETFA, 2023, pp. 1–8. doi:10.1109/ETFA54631.2023.10275411.
- [14] H. Koziolok, et al., LLM-based and retrieval-augmented control code generation, in: Proc. LLM4Code Workshop, 2024, pp. 22–29. doi:10.1145/3643795.3648384.
- [15] M. Fakihi, et al., LLM4PLC: Harnessing large language models for verifiable programming of PLCs in industrial control systems, in: Proc. ICSE-SEIP, 2024. doi:10.1145/3639477.3639743.
- [16] Z. Liu, et al., Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents, 2024. arXiv:2410.14209.
- [17] D. Darvas, E. Blanco Viñuela, B. Fernández Adiego, PLCverif: A tool to verify PLC programs based on model checking techniques, in: Proc. ICALEPCS, 2015. doi:10.18429/JACoW-ICALES2015-WEPGF092.
- [18] T. Ovatman, et al., An overview of model checking practices on verification of PLC software, Software & Systems Modeling 15 (2016) 937–960. doi:10.1007/s10270-014-0448-7.
- [19] E. J. Hu, et al., LoRA: Low-rank adaptation of large language models, in: Proc. ICLR, 2022. arXiv:2106.09685.

- [20] N. Houlsby, et al., Parameter-efficient transfer learning for NLP, in: Proc. ICML, 2019. [arXiv:1902.00751](#).
- [21] T. Dettmers, et al., QLoRA: Efficient finetuning of quantized LLMs, in: Proc. NeurIPS, 2023. [arXiv:2305.14314](#).
- [22] DeepSeek-AI, DeepSeek-V3 technical report, 2024. [arXiv:2412.19437](#).
- [23] Y. Wang, et al., Self-Instruct: Aligning language models with self-generated instructions, in: Proc. ACL, 2023, pp. 13484–13508. doi:10.18653/v1/2023.acl-long.754.
- [24] Z. Luo, et al., WizardCoder: Empowering code large language models with Evol-Instruct, in: Proc. ICLR, 2024. [arXiv:2306.08568](#).
- [25] Y. Wei, et al., Magicoder: Empowering code generation with OSS-Instruct, in: Proc. ICML, 2024. [arXiv:2312.02120](#).
- [26] P. Yin, G. Neubig, A syntactic neural model for general-purpose code generation, in: Proc. ACL, 2017, pp. 440–450. doi:10.18653/v1/P17-1041.
- [27] M. Rabinovich, M. Stern, D. Klein, Abstract syntax networks for code generation and semantic parsing, in: Proc. ACL, 2017, pp. 1139–1149. doi:10.18653/v1/P17-1105.
- [28] T. Scholak, N. Schucher, D. Bahdanau, PICARD: Parsing incrementally for constrained autoregressive decoding from language models, in: Proc. EMNLP, 2021, pp. 9895–9901. doi:10.18653/v1/2021.emnlp-main.779.
- [29] G. Poesia, et al., Synchromesh: Reliable code generation from pre-trained language models, in: Proc. ICLR, 2022. [arXiv:2201.11227](#).
- [30] S. Geng, et al., Grammar-constrained decoding for structured NLP tasks without finetuning, in: Proc. EMNLP, 2023, pp. 10932–10952. doi:10.18653/v1/2023.emnlp-main.674.
- [31] B. T. Willard, R. Louf, Efficient guided generation for large language models, 2023. [arXiv:2307.09702](#).
- [32] X. Chen, et al., Teaching large language models to self-debug, in: Proc. ICLR, 2024. [arXiv:2304.05128](#).
- [33] A. Madaan, et al., Self-Refine: Iterative refinement with self-feedback, in: Proc. NeurIPS, 2023. [arXiv:2303.17651](#).
- [34] N. Shinn, et al., Reflexion: Language agents with verbal reinforcement learning, in: Proc. NeurIPS, 2023. [arXiv:2303.11366](#).
- [35] T. X. Olausson, et al., Is self-repair a silver bullet for code generation?, in: Proc. ICLR, 2024. [arXiv:2306.09896](#).
- [36] J. Huang, et al., Large language models cannot self-correct reasoning yet, in: Proc. ICLR, 2024. [arXiv:2310.01798](#).
- [37] Y. Li, et al., Competition-level code generation with AlphaCode, Science 378 (2022) 1092–1097. doi:10.1126/science.abq1158.
- [38] B. Chen, et al., CodeT: Code generation with generated tests, in: Proc. ICLR, 2023. [arXiv:2207.10397](#).
- [39] A. Ni, et al., LEVER: Learning to verify language-to-code generation with execution, in: Proc. ICML, 2023. [arXiv:2302.08468](#).
- [40] J. Ferrante, K. J. Ottenstein, J. D. Warren, The program dependence graph and its use in optimization, ACM Trans. Program. Lang. Syst. 9 (1987) 319–349. doi:10.1145/24039.24041.
- [41] S. Ren, et al., CodeBLEU: a method for automatic evaluation of code synthesis, 2020. [arXiv:2009.10297](#).
- [42] D. Guo, et al., GraphCodeBERT: Pre-training code representations with data flow, in: Proc. ICLR, 2021. [arXiv:2009.08366](#).
- [43] J. Liu, et al., Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation, in: Proc. NeurIPS, 2023. [arXiv:2305.01210](#).
- [44] Modbus Organization, MODBUS application protocol specification V1.1b3, 2012. URL: <https://www.modbus.org/file/secure/modbusprotocolspecification.pdf>.